

PROOF-CARRYING DEBUGGER

Buggy

Usage & Sandbox Guide — how to run it on any project,
real TypeScript and Python RAG-chatbot walkthroughs, and how the execution sandbox
works

Version 0.1.0 · MIT License · Node $\geq$ 18
TypeScript / JavaScript (full) · Python (bug proving)

1. What Buggy is

Buggy is a multi-agent **proof-carrying debugger**. Instead of flagging code that "looks risky," it runs a four-stage pipeline that finds a concrete input, proves the bug reproduces, proposes candidate fixes, and rejects fixes that only paper over the failing test.

Parse → **Prove** → **Repair** → **Classify**, coordinated by an orchestrator, and exposed three ways:

- **CLI** — `buggy` for local/CI use.
- **MCP server** — `buggy-mcp`, so AI IDEs (Kiro, Cursor, Windsurf) call it as tools.
- **Programmatic API** — the `ProofDebugger` class for scripts and integrations.

It also ships a Kiro integration (hooks + steering) and an **experience memory ("Watchlist")** that records what worked and what failed on every investigation, so results improve with each iteration.

What makes it different: a *proof-of-failure certificate* (a real triggering input, the faulty output, and the violated postcondition, verified for admissibility, soundness, and reproducibility) plus an *overfitting classifier* that throws out patches which only match the test.

2. Language support (read this first)

TypeScript & JavaScript: fully supported (parse, prove, repair, classify, analyze).

Python: bug proving supported — `investigate` proves bugs in `.py` files by executing them. CST features (`analyze` / `list_functions`) need an optional grammar (below).

How Python proving works without a grammar

The prover reads the target function's source and **runs it in the system Python interpreter** to search for failing inputs. It does not need a parser for that, so proving Python is available immediately. Set the language in `.debugger.yaml` :

```
language: python
sandbox: { runtime: python, timeout_seconds: 30 }
```

Execution is routed by language: Python targets run in a `python` subprocess, TS/JS targets in a Node subprocess (see §6).

Prove / investigate	TS, JS, Python — executes the function, catches crashes (incl. Python <code>ZeroDivisionError</code> , <code>IndexError</code> , <code>TypeError</code>), timeouts, NaN/Infinity, non-determinism, and postcondition violations.
Analyze / list_functions / graph	TS, JS today. For Python these need the Tree-sitter Python grammar. The parser loads <code>tree-sitter-python</code> automatically if present; if absent it falls back to the TypeScript grammar (so nothing breaks, but Python CST results are unreliable).
Postcondition syntax	Conditions (e.g. <code>isFinite(result)</code> , <code>result >= 0</code>) are written as JavaScript expressions and evaluated against the returned value regardless of target language.

Enabling Python *parsing* (analyze)

Install the grammar once, in an environment with npm registry access:

```
npm install tree-sitter-python --legacy-peer-deps
```

No code change is needed — the parser detects and uses it on the next run. (In networks with TLS inspection you may need your corporate CA configured for npm to reach the registry.)

For a RAG chatbot: TypeScript stacks (LangChain.js, LlamaIndex.TS, Vercel AI SDK) get the full pipeline. Python stacks (LangChain, LlamaIndex, FastAPI) get bug proving now, and full parsing/analyze once the grammar is installed.

3. Install & set up on any project

```
# 1. Install
npm install buggy          # or: npm install -g buggy for the CLI

# 2. Initialize in your project root
cd /path/to/your/project
npx buggy init             # creates .debugger.yaml and .debugger/

# 3. Ignore the working directory
echo ".debugger/" >> .gitignore
```

Configure `.debugger.yaml`

```
language: typescript      # or: python
parser:  { command: tree-sitter-typescript } # tree-sitter-python for Python
lsp:     { command: typescript-language-server } # optional
sandbox: { runtime: node, memory_limit_mb: 512, timeout_seconds: 60 } # runtime: python for Python
oracles: { timeout_threshold_seconds: 10, crash_detection: true,
          overflow_detection: true, determinism_check_count: 5 }
probe:   { search_budget: 100, max_refinement_iterations: 10 }
watchlist: { enabled: true, scope: layered } # experience memory (see §8)
```

The `watchlist.scope` defaults to `layered`: raw episodes stay local, verified lessons are shared with your team via committed steering, and generalized lessons corroborated across ≥ 2 projects promote to a personal global store. Set `scope: local` for sensitive repos.

4. The pipeline, stage by stage

Stage	What happens	Output
Parse	Fault-tolerant Tree-sitter CST, optional LSP symbol resolution, call graph.	CST + errors, graph in SQLite
Prove	Backward slice, spec refinement (PROBE loop), specification-aware fuzzing that runs the function (Node or Python) on generated inputs looking for a postcondition violation or crash.	Proof-of-failure certificate, or <code>unconfirmed</code>
Repair	AST-aware candidate patches from multiple strategies (guards, expression/return correction, etc.).	≥ 3 candidate patches
Classify	66-dimension AST-difference vector scored for overfitting; approve below threshold, reject above.	Approved / rejected patches

Terminal statuses: `confirmed_and_repaired` (bug proven + at least one non-overfit fix), `confirmed_no_repair` (bug proven, all fixes rejected), `unconfirmed` (no bug proven), `halted` .

5. Walkthrough: a RAG chatbot

A retrieval-augmented chatbot lives or dies on its retrieval math. The classic production incidents are silent: a NaN/undefined similarity from an empty embedding, a crash on an empty document set, an infinite loop in a chunker. Below is Buggy run against RAG retrieval math in **both TypeScript and Python**. Every result is actual output.

TypeScript (embeddings.ts)

```
export function cosineSimilarity(a: number[], b: number[]): number {
  let dot = 0, normA = 0, normB = 0;
  for (let i = 0; i < a.length; i++) { dot += a[i]*b[i]; normA += a[i]**2; normB += b[i]**2; }
  return dot / (Math.sqrt(normA) * Math.sqrt(normB)); // zero vector -> 0/0 = NaN
}
```

```
buggy investigate cosineSimilarity --file src/embeddings.ts
```

```
cosineSimilarity  status=confirmed_no_repair
proof.test_input  = [[], []]                # two empty embedding vectors
observed_output   = NaN
violated          = !isNaN(result)
patches           = 0 approved, 3 rejected (all flagged overfit)
```

```
averageEmbeddings status=confirmed_no_repair
proof.test_input   = [[]]                    # empty embedding set
violated           = TypeError: Cannot read properties of undefined (reading 'length')
```

Python (embeddings.py)

```
import math

def cosine_similarity(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(y * y for y in b))
    return dot / (norm_a * norm_b) # zero vector -> ZeroDivisionError

def average_embeddings(vectors):
    dim = len(vectors[0]) # [] -> IndexError
    ...
```

```
buggy investigate cosine_similarity --file src/embeddings.py # .debugger.yaml: language: python
```

```
cosine_similarity  status=confirmed_no_repair
proof.test_input   = [[], []]                # two empty embedding vectors
violated           = function must not throw: ZeroDivisionError: division by zero
```

```
average_embeddings status=confirmed_no_repair
proof.test_input   = [[]]                    # empty embedding set
violated           = function must not throw: IndexError: list index out of range
```

Same guarantee, both languages: before your chatbot ships, Buggy hands you the exact inputs that break retrieval — proven by executing the real function. Note the language-specific behavior it captures faithfully: JavaScript's empty-vector similarity yields `NaN`, while Python's raises `ZeroDivisionError`.

Honest note on the fixes: in these cases every candidate patch was classified as *overfit* and rejected (`confirmed_no_repair`). Buggy is deliberately conservative: it would rather hand you a proven bug than auto-apply a fix that only satisfies the one failing input. You then write the guard (e.g. `return 0` for a zero-norm vector) with the exact trigger in hand.

The memory makes the next run smarter

```
tool: buggy_recall args: { "function_id": "cosine_similarity" }
-> { "seen_before": true,
    "lessons": [ { "tier": "project", "title": "... in cosine_similarity",
                  "what_failed": ["Overfitting risk: ..."], "trigger_shape": "[[],[]]" } ] }
```

With the *recall-first* hook installed, the next time anyone asks Kiro to touch `cosine_similarity`, Kiro is reminded of the proven failure at `[[],[]]` and which fix shapes were rejected — so it writes the zero-vector guard directly.

6. The execution sandbox — how it actually works

To prove a bug, Buggy must *run* your function on generated inputs. Execution is routed by language to a matching runtime, both under the same contract.

TypeScript / JavaScript	A forked Node.js child process running a generated runner; TypeScript is stripped to JavaScript; results returned over Node IPC .
Python	A spawned python child process running a generated runner that execs the module and calls the target; results returned via a temp result file (Python has no Node IPC), keeping the function's own stdout from corrupting the payload.
Timeout	A parent-managed wall-clock timeout (from <code>oracles.timeout_threshold_seconds / sandbox.timeout_seconds</code>). On timeout the child is <code>SIGKILL</code> ed — this is how infinite-loop bugs are caught.
Scratch space	Runner scripts and payloads are written under the OS temp dir (<code><tmp>/buggy-exec</code> , <code><tmp>/buggy-pyexec</code>) and deleted after each run.
Edge-value fidelity	<code>NaN</code> , <code>Infinity</code> , <code>-Infinity</code> , and <code>undefined / None</code> are preserved through sentinels in both directions, so the very edge cases that cause bugs are tested faithfully.
Determinism check	The function can be run N times on the same input to distinguish real bugs from flakes.

Oracles (what counts as a failure)

- **Crash** — an uncaught exception (JS `TypeError` ; Python `ZeroDivisionError` , `IndexError` , `TypeError` , ...).
- **Timeout** — exceeds the wall-clock budget (infinite loop / pathological input).
- **Postcondition violation** — output breaks a declared spec (e.g. `!isNaN(result)`).
- **Non-determinism** — differing outputs across identical runs.
- **Overflow** — integer/numeric overflow detection.

Isolation reality — important for security claims. Both runtimes use **process-level isolation, not VM-level**. The child (Node or Python) shares the host filesystem, network, and user privileges of the parent process. Memory limits are *reported, not OS-enforced*. This is fine for analyzing *your own* code. Do **not** point it at untrusted third-party code expecting containment, and do not market it as "hardware isolation."

Firecracker microVM — designed, not yet active. The codebase includes a microVM isolation layer (per-VM memory caps, network egress policy, syscall allow-list, OAP execution passports, a circuit breaker, and a snapshot pool). It is **not wired into the default pipeline**; the subprocess executors are used instead. Treat microVM isolation as roadmap, and enable/validate it before making isolation guarantees.

7. Interface reference

CLI

Command	Purpose
<code>buggy init</code>	Create <code>.debugger.yaml</code> + <code>.debugger/</code> .
<code>buggy analyze <file></code>	Parse a file; list functions and syntax errors.
<code>buggy investigate <fn> --file <path></code>	Run the full pipeline on a function (TS/JS/Python).
<code>buggy status <id></code>	Progress of a running/finished investigation.
<code>buggy halt <id></code>	Stop an investigation, keep partial results.

Global flags: `--json` (machine-readable), `--verbose`.

MCP tools (7)

Tool	Purpose
<code>buggy_init</code>	Initialize the debugger for a project.
<code>buggy_analyze</code>	Parse a file, return CST analysis + function list.
<code>buggy_investigate</code>	Run the full Parse→Prove→Repair→Classify pipeline.
<code>buggy_status</code>	Investigation status.
<code>buggy_query_graph</code>	Query the semantic graph (callees, node, file graph).
<code>buggy_list_functions</code>	List function declarations in a file.
<code>buggy_recall</code>	Recall prior Watchlist lessons before editing/fixing code.

API (ProofDebugger)

`initialize`, `parse`, `investigate`, `getStatus`, `halt`, `queryCallees`, `queryNode`, `queryFileGraph`, `getConfig`, `recall`, `watchlistStats`, `shutdown`.

8. Watchlist — the experience memory

Every investigation writes one **episode** (what was attempted, what worked, what failed and how). Verified, proof-backed episodes distill into **lessons** that are fed back before the next change.

Scope	Lives in	Shared with
local	<code>.debugger/</code> (git-ignored)	nobody — raw episodes
team	<code>.kiro/steering/buggy-watchlist.md</code> (committed)	teammates on the repo
global	<code>~/.buggy/</code> + user-level steering	all your projects (sanitized, ≥ 2 -project corroboration)

- **Recall:** `buggy_recall` and `ProofDebugger.recall()` merge project + global lessons, project winning on conflict.
- **Safety:** only proof-backed lessons promote; the global tier is sanitized of code, paths, and literal values; investigations never modify your source files.
- **Kiro:** the *recall-first* hook consults the memory before edits; committed team steering means teammates inherit every lesson.

9. Limitations to represent honestly

Area	Reality today
Languages — proving	TypeScript, JavaScript, and Python are executed and proven (verified end-to-end).
Languages — analyze/CST	TS/JS today. Python <code>analyze / list_functions</code> require installing <code>tree-sitter-python</code> (the parser auto-detects it); until then Python CST features fall back to the TS grammar and are unreliable. Rust/Go/others are not wired.
Isolation	Process-level (forked Node / spawned Python), not VM-level. Firecracker microVM is present but inactive (§6).
Auto-fix	Proposes and screens fixes; frequently rejects all candidates as overfit, yielding <code>confirmed_no_repair</code> . Best framed as "proves bugs and screens fixes," not "auto-fixes."
Patch validation	The compile/emulate/test filtering pipeline exists but is not on the default investigate path; candidates are screened for overfitting, not compile/test-verified.
Best-fit inputs	Scalar/numeric functions prove most readily; array/object inputs may need tighter preconditions or return <code>unconfirmed</code> .

Rock-solid today: TS/JS parsing, bug proving with reproducing certificates across TS/JS/Python, overfitting classification, the CLI / MCP / API surfaces, the Kiro hooks & steering, and the full Watchlist memory — all test-backed (900+ passing tests).

Buggy v0.1.0 · Usage & Sandbox Guide. Examples in §5 are real output from the bundled RAG demos (TypeScript `live-demo/rag-pipeline`; Python equivalents executed via the Python runtime). Language and sandbox statements reflect the current implementation, not aspirational documentation.